CS 61B Fall 2025

Midterm 2 Thursday, November 6, 2025

Print Your Name: _____

Print Your Student ID: _____

Print Student name to your left: _____

Print Student name to your right: _____

You have 110 minutes. There are 7 questions of varying credit. (100 points total)

Question:	1	2	3	4	5	6	7	Total
Points:	15	16	15	15	9	16	14	100

For questions with **circular bubbles**, select only one choice (there is only one correct answer).

- ○ Unselected option (completely unfilled)
- ⊘ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square boxes**, you may select one or more choices (select all that apply).

- ■ You can select
- ■ multiple squares
- ☑ Don't do this (it will be graded as incorrect)

- Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers or your answer is ambiguous, we will grade the **worst** interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) or a Java library not on the reference card (e.g. Google Truth) is relevant, it will be explicitly noted in the question.
- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B/61BL style guidelines. For example, any method, constructor, or if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

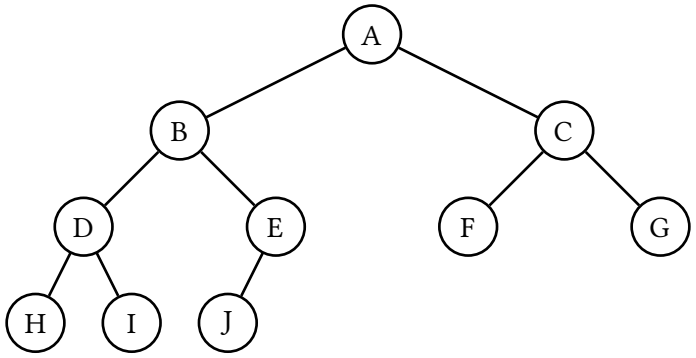
Read the honor code below and sign your name.

> By signing below, I affirm that all work on this exam is my own work. I have not referenced any disallowed materials, nor collaborated with anyone else on this exam. I understand that if I cheat on the exam, I may face the penalty of an "F" grade and a referral to the Center for Student Conduct.

Sign your name: _____

Q1 *Treevia* **(15 points)**

Consider the tree below. The letters represent unknown integer values, and all 10 values are **unique**.



Q1.1 (3 points) Suppose the tree represents a max heap. Which could be the third-largest value?

☐ A ☐ B ☐ C ☐ D ☐ E ☐ F ☐ G ☐ H ☐ I ☐ J

Q1.2 (2 points) Suppose the tree represents a binary search tree. Which could be the third-largest value?

☐ A ☐ B ☐ C ☐ D ☐ E ☐ F ☐ G ☐ H ☐ I ☐ J

Q1.3 (2 points) Suppose the tree represents a binary search tree. We delete A using Hibbard deletion. Which nodes could become the new root?

☐ B ☐ C ☐ D ☐ E ☐ F ☐ G ☐ H ☐ I ☐ J

Q1.4 (2 points) Suppose we perform an in-order tree traversal of the subtree rooted in B. Which will be the last node visited?

○ B ○ D ○ E ○ H ○ I ○ J

Q1.5 (2 points) The statement "F is strictly greater than J" is always true if the tree represents a _____.

Select all options that could go in the blank.

☐ Min heap ☐ Max heap ☐ BST ☐ None

Q1.6 (2 points) Suppose we treat the tree as an undirected graph, and run a breadth first search starting from G. Which could be the last node visited?

☐ A ☐ B ☐ C ☐ D ☐ E ☐ F ☐ H ☐ I ☐ J ☐ None

Q1.7 (2 points) Suppose the tree represents a binary search tree, and we want to turn the tree into an LLRB. Select a set of edges to turn red such that the LLRB is valid.

☐ A–B ☐ B–D ☐ C–F ☐ D–H ☐ E–J

☐ None, the tree is already a valid LLRB with all black links.

Q2 *Aditya's Potpourri* **(16 points)**

For Q2.1 and Q2.2, choose whether the following arrays represent a max heap, min heap, or neither:

Q2.1 (1 point) `[-, 1, 7, 2, 8, 15, 5, 3, 17, 9]`

◯ Min heap ◯ Max heap ◯ Neither

Q2.2 (1 point) `[-, 59, 38, 47, 26, 46, 45, 13]`

◯ Min heap ◯ Max heap ◯ Neither

Q2.3 (2 points) From a pool of $n > 75$ unique integers, Aditya wants to find the 75th-smallest element. Which data structure would be best for this task?

◯ Hash table ◯ Linked list

◯ Max heap ◯ Binary search tree

Q2.4 (1 point) True or False: If `hashCode()` is a valid hash function, then `hashCode() + c` is also a valid hash function, where `c` is a fixed integer in $[-1000000, 1000000]$.

◯ True ◯ False

Q2.5 (1 point) True or False: If `hashCode()` is a valid hash function, then `hashCode() * c` is also a valid hash function, where `c` is a fixed integer in $[-1000000, 1000000]$.

◯ True ◯ False

Q2.6 (2 points) True or False: Finding a value is asymptotically faster in a 2-3 tree than in an LLRB, because a 2-3 tree has a shorter height.

◯ True ◯ False

Q2.7 (2 points) Suppose we have a valid LLRB, and we add an element which creates a right-leaning red link. With no other information, select **all** possible operations that could be the first balancing operation after the insertion.

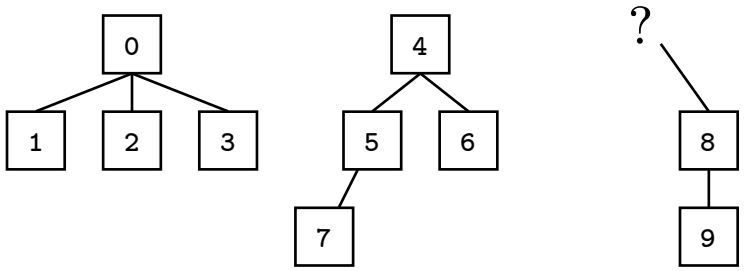
☐ `rotateLeft()` ☐ `rotateRight()` ☐ `colorFlip()`

Q2.8 (0 points) What is the name of the largest known star?

Q2.9 (3 points) Josh built a valid Weighted Quick Union (without path compression) with 10 items. However, he lost one of the values in the parents array!

`[-1, 0, 0, 0, -1, 4, 4, 5, ?, 8]`

(Note: This array uses `-1` instead of `-size` for roots.)



Assuming `8` is not a root, select all items that could be the parent of `8`.

☐ `0` ☐ `1` ☐ `2` ☐ `3` ☐ `4` ☐ `5` ☐ `6` ☐ `7`

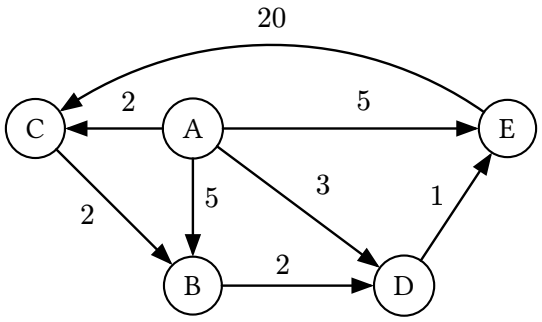
Q2.10 (3 points) We are building a Weighted Quick Union (without path compression), and the object initially has zero connections.

What is the minimum number of **`union`** operations needed to create a tree of height 3?

Recall that a tree's height is the number of links from root to bottommost leaf. For example, a freshly initialized WQU object has a height of 0, not 1.

◯ 3 ◯ 5 ◯ 7 ◯ 15 ◯ 16 ◯ 31 ◯ 32

Q3 *Graphery* **(15 points)**



Suppose we run Dijkstra's on the graph above, starting at vertex A, tiebreaking by alphabetical order.

Q3.1 (2 points) In what order will the vertices be visited?

◯ A, C, D, B, E ◯ A, D, E, C, B ◯ A, B, C, D, E ◯ A, B, D, E, C

Q3.2 (2 points) Which vertices have their `distTo` value updated the most number of times?

☐ A ☐ B ☐ C ☐ D ☐ E

Q3.3 (2 points) Suppose we replace the edge weight for A → B from 5 to −1000. Will running Dijkstra's from A still correctly return a shortest paths tree?

◯ Yes ◯ No

Q3.4 (3 points) Suppose we instead replace the edge weight E → C. If this edge weight gets too small, there would no longer be a finite shortest path from A to every vertex.

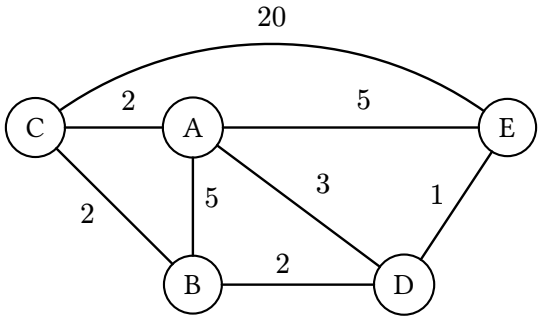
What is the smallest weight for edge E → C such that there is still a finite shortest path from A to every vertex?

Q3.5 (2 points) Which edge, if reversed, causes the resulting graph to have a valid topological sort?

◯ A → B ◯ A → C ◯ A → D ◯ A → E ◯ C → B

◯ The graph already has a valid topological sort.

Consider the same graph, but with undirected edges.



For Q3.6 to Q3.9, run Prim's algorithm from vertex A, tiebreaking alphabetically.

Q3.6 (0.5 points) First edge added to the MST:

◯ A-B ◯ A-C ◯ A-D ◯ A-E ◯ B-C ◯ B-D ◯ C-E ◯ D-E

Q3.7 (0.5 points) Second edge added to the MST:

◯ A-B ◯ A-C ◯ A-D ◯ A-E ◯ B-C ◯ B-D ◯ C-E ◯ D-E

Q3.8 (0.5 points) Third edge added to the MST:

◯ A-B ◯ A-C ◯ A-D ◯ A-E ◯ B-C ◯ B-D ◯ C-E ◯ D-E

Q3.9 (0.5 points) Fourth edge added to the MST:

◯ A-B ◯ A-C ◯ A-D ◯ A-E ◯ B-C ◯ B-D ◯ C-E ◯ D-E

Q3.10 (2 points) For the graph above, the MST outputted by Kruskal's algorithm is ____ the same as the MST outputted by Prim's algorithm (from Q3.6 to Q3.9).

◯ Always ◯ Sometimes (depends on tiebreaking) ◯ Never

Q4 *Project 4E* **(15 points)**

Project 4 ignored the definitions of words, but Aditya likes definitions. Implement the `WordDefinitions` class below, such that `getDefinitions` returns all definitions of a word. You may not need all lines.

Reminder: `synsetsFile` contains definitions for each synset. Here are some example lines from the file:

```
25,chill iciness,coldness from environment
27,chill shivering,cold feeling
```

Examples based on the lines above:

- `getDefinitions("chill")` should return `["coldness from environment", "cold feeling"]`.
- `getDefinitions("shivering")` should return `["cold feeling"]`.

Hint: `split(String token)` divides a string into an array of substrings on each appearance of `token`.

```
public class WordDefinitions {

    public _____ definitions;

    public WordDefinitions(String synsetsFile) {

        definitions = _____;

        In synsetIn = new In(synsetsFile);

        while (synsetIn.hasNextLine()) {
            String line = synsetIn.readLine();

            _____ splitLine = _____.split(_____);

            _____ splitWords = _____;

            for (_____ : _____) {

                if (_____ == null) {

                    _____;

                    _____;
                }

                _____;

                _____;
            }
        }
    }

    /* Assume word exists in the synsets file. */
    public List<String> getDefinitions(String word) {

        _____;

    }
}
```

Q5 *Hash Table* **(9 points)**

Consider the class below:

```
public class MutableString {
    public String str;

    public MutableString(String s) {
        this.str = s;
    }

    /* Changes the idx'th character of str to c. */
    public void changeChar(int idx, char c) { ... }

    /* Adds character c to the end of str. */
    public void addChar(char c) { ... }

    @Override
    public boolean equals(Object other) {
        if (other instanceof MutableString uddaMutableString) {
            return this.str.equals(uddaMutableString.str);
        }
        return false;
    }

    @Override
    public int hashCode() {
        return this.str.length();
    }
}
```

Suppose we insert four items into a `HashSet` of `MutableString` objects.

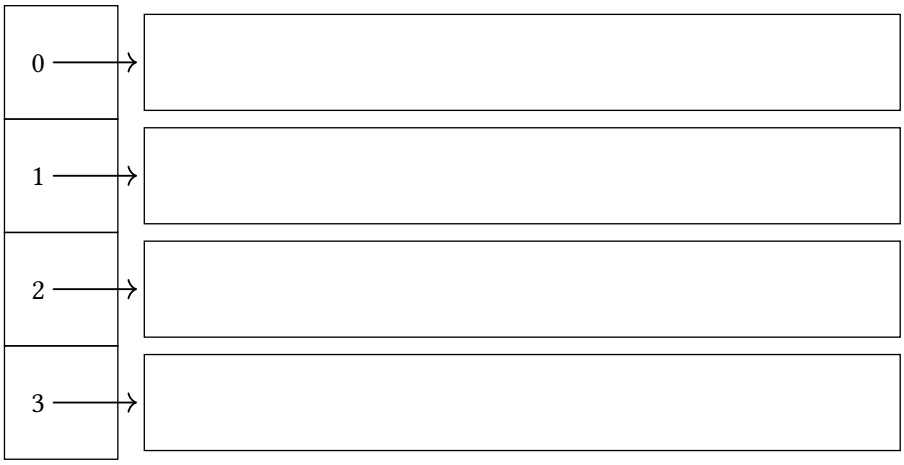
```
Set<MutableString> h = new HashSet<>();

MutableString s1 = new MutableString("cat");
h.add(s1);

h.add(new MutableString("fish"));
h.add(new MutableString("dog"));
h.add(new MutableString("horse"));
```

Q5.1 (3 points) Assuming the hash table has four buckets, draw the results of the insertions above in the hash table. Do not worry about resizing.

You may refer to `MutableString` objects by the value of their `str` field. In each box, write a list like `["cat", "fish", "dog", "horse"]`, or `[ ]` if the bucket is empty.



Q5.2 (3 points) After the four insertions, we run `s1.changeChar(0, 'b')`, changing `s1.str` from `"cat"` to `"bat"`.

Which of the following are true?

- ☐ `h.contains(s1)` returns `true`.
- ☐ `h.contains(new MutableString("bat"))` returns `true`.
- ☐ `h.contains(new MutableString("cat"))` returns `true`.
- ☐ None of the above

Q5.3 (3 points) Consider the hash table from Q5.1, i.e. we have **not** changed `"cat"` to `"bat"`.

Suppose we run `s1.addChar('s')`, changing `s1.str` from `"cat"` to `"cats"`.

Which of the following are true?

- ☐ `h.contains(s1)` returns `true`.
- ☐ `h.contains(new MutableString("cat"))` returns `true`.
- ☐ `h.contains(new MutableString("cats"))` returns `true`.
- ☐ `h.add(new MutableString("cats"))` adds a new element to our hash table.
- ☐ None of the above

Q6 *Asymptotics Analysis* **(16 points)**

Q6.1 (4 points) What is the runtime of `mulch` in terms of N?

```
void mulch(int[] x) {
    int N = x.length;
    if (N <= 1) {
        return;
    }

    int[] left = new int[N/2];
    for (int i = 0; i < N/2; i += 1) {
        left[i] = x[i];
    }

    mulch(left);
    mulch(left);
}
```

Best case:

◯ $\Theta(1)$	◯ $\Theta(\log(\log N))$	◯ $\Theta(\log N)$	◯ $\Theta((\log N)^2)$	◯ $\Theta\left(\sqrt{N}\right)$
◯ $\Theta(N)$	◯ $\Theta(N \log N)$	◯ $\Theta(N^2)$	◯ $\Theta(N^2 \log N)$	◯ $\Theta(N^3)$
◯ $\Theta(N^3 \log N)$	◯ $\Theta(2^N)$	◯ $\Theta(N!)$	◯ $\Theta(N^N)$	◯ Infinite loop

Worst case:

◯ $\Theta(1)$	◯ $\Theta(\log(\log N))$	◯ $\Theta(\log N)$	◯ $\Theta((\log N)^2)$	◯ $\Theta\left(\sqrt{N}\right)$
◯ $\Theta(N)$	◯ $\Theta(N \log N)$	◯ $\Theta(N^2)$	◯ $\Theta(N^2 \log N)$	◯ $\Theta(N^3)$
◯ $\Theta(N^3 \log N)$	◯ $\Theta(2^N)$	◯ $\Theta(N!)$	◯ $\Theta(N^N)$	◯ Infinite loop

Q6.2 (4 points) What is the runtime of `alarmJunior` in terms of N?

```
void alarmJunior(int N) {
    alarmHelper(2, N);
}

void alarmHelper(int i, int N) {
    if (i > N) {
        return;
    }

    System.out.println("BEEP");
    alarmHelper(i*2, N/2);
}
```

◯ $\Theta(1)$	◯ $\Theta(\log(\log N))$	◯ $\Theta(\log N)$	◯ $\Theta((\log N)^2)$	◯ $\Theta\left(\sqrt{N}\right)$
◯ $\Theta(N)$	◯ $\Theta(N \log N)$	◯ $\Theta(N^2)$	◯ $\Theta(N^2 \log N)$	◯ $\Theta(N^3)$
◯ $\Theta(N^3 \log N)$	◯ $\Theta(2^N)$	◯ $\Theta(N!)$	◯ $\Theta(N^N)$	◯ Infinite loop

Q6.3 (4 points) Consider the **BSTMap** from Homework 9. Recall that **get** returns **null** if the item is not in the map.

What is the runtime of **boom** in terms of N, where $N =$ **map.size()**?

Note: The runtime of the **compareTo** method of the **Laboohoo** class is constant.

```
void boom(BSTMap<Integer, Laboohoo> map) {
    for (int i = 0; i < map.size(); i += 1) {
        System.out.println(map.get(i));
    }
}
```

Best case:

◯ $\Theta(1)$	◯ $\Theta(\log(\log N))$	◯ $\Theta(\log N)$	◯ $\Theta((\log N)^2)$	◯ $\Theta\left(\sqrt{N}\right)$
◯ $\Theta(N)$	◯ $\Theta(N \log N)$	◯ $\Theta(N^2)$	◯ $\Theta(N^2 \log N)$	◯ $\Theta(N^3)$
◯ $\Theta(N^3 \log N)$	◯ $\Theta(2^N)$	◯ $\Theta(N!)$	◯ $\Theta(N^N)$	◯ Infinite loop

Worst case:

◯ $\Theta(1)$	◯ $\Theta(\log(\log N))$	◯ $\Theta(\log N)$	◯ $\Theta((\log N)^2)$	◯ $\Theta\left(\sqrt{N}\right)$
◯ $\Theta(N)$	◯ $\Theta(N \log N)$	◯ $\Theta(N^2)$	◯ $\Theta(N^2 \log N)$	◯ $\Theta(N^3)$
◯ $\Theta(N^3 \log N)$	◯ $\Theta(2^N)$	◯ $\Theta(N!)$	◯ $\Theta(N^N)$	◯ Infinite loop

Q6.4 (4 points) When Edsger Dijkstra published his original version of Dijkstra's algorithm, he did not use any special data structure to store the fringe.

Instead, he simply scanned the **distTo** array, choosing the vertex with the smallest value that was not yet **marked** as previously visited.

Give a tight O bound, in terms of V and/or E, for the runtime of this algorithm on a graph with V vertices and E edges. Simplify your answer.

Assume we are using an adjacency list.

$O($ $)$

Q7 BorBders (14 points)

Bob loves walking between countries, which are represented by the class below:

```
public class Country {
    public String name;           // Name of the country.

    public List<Country> borders; // A list of bordering countries.
                                  // borders is never null.
}
```

Borders go both ways, i.e. if `countryA` appears in `countryB`'s `borders`, then `countryB` appears in `countryA`'s `borders`.

Implement the `WalkManager` class, which has two methods:

- `public WalkManager(List<Country> countries)`:
  Constructor. Given the list of all N countries in the world, prepare any useful instance variables.
  Must run in $O(N^2 \log N)$ time.

- `public boolean canWalk(Country a, Country b)`:
  Returns whether country `a` is reachable from country `b`, using one or more borders.
  Must run in $O(\log N)$ time.

Here is an example in a world that has $N = 4$ countries:
- `USA` borders `[Canada, Mexico]`.
- `Canada` borders `[USA]`.
- `Mexico` borders `[USA]`.
- `Ethanland` borders `[]` (empty list).

```
WalkManager wm = new WalkManager(List.of(USA, Canada, Mexico, Ethanland));

// true: Canada to USA to Mexico.
wm.canWalk(Canada, Mexico);

// false: No sequence of borders exists between Ethanland and USA.
wm.canWalk(Ethanland, USA);
```

The skeleton code begins on the next page.

Implement **WalkManager** according to Bob's requirements.

Reminder: Everything on the reference card has been imported, and all data structures behave according to their implementations in lecture. You may not need all lines.

```
public class WalkManager {
    // Add any instance variables below (if necessary)

    _____;

    _____;

    public WalkManager(List<Country> countries) {

        int N = countries.size();

        _____;

        _____;

        for (int i = 0; i < _____; i += 1) {

            Country curr = _____;

            _____;
        }

        for (_____) {

            for (Country c : _____) {

                _____;
            }
        }
    }

    public boolean canWalk(Country a, Country b) {

        return _____;
    }
}
```

You have reached the end of the exam! Nothing on this page is worth any credit.

Playing Favorites

CS61B staff has a collective favorite data structure! What do you think it is?

Feedback

Leave any feedback, comments, and/or drawings in the box below!